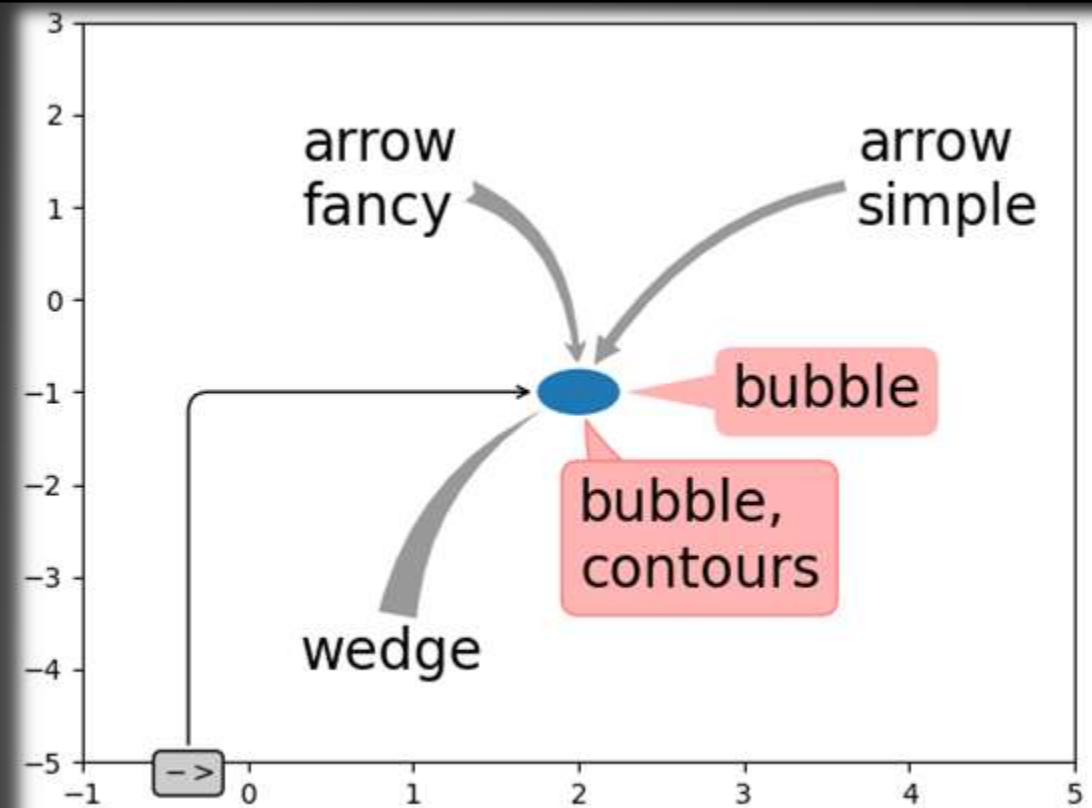


TEXT AND ANNOTATION

- TRANSFORMS AND TEXT POSITION
- ARROWS AND ANNOTATION



TEXT AND ANNOTATION

- A good visualization involves guiding the reader so that the figure tells a story.
- This story can be told in an entirely visual manner, without the need for added text (or) small textual cues and labels are necessary.
- The most basic types of annotations used are axes labels and titles.
- Data can be visualized and annotated to help convey interesting information.

```
import matplotlib.pyplot as plt
import matplotlib as mpl
plt.style.use('seaborn-whitegrid')
import numpy as np
import pandas as pd
```

TEXT AND ANNOTATION

- The 'text' method → takes an x position, a y position, a string, and then optional keywords specifying the color, size, style, alignment, and other properties of the text.

```
import matplotlib.pyplot as plt
import numpy as np
plt.text(2, 9, 'Hello, World!', fontsize=12, color='orange', ha='center')
plt.xlim(0, 10)
plt.ylim(0, 10)
plt.show()
```



TRANSFORMS AND TEXT POSITION

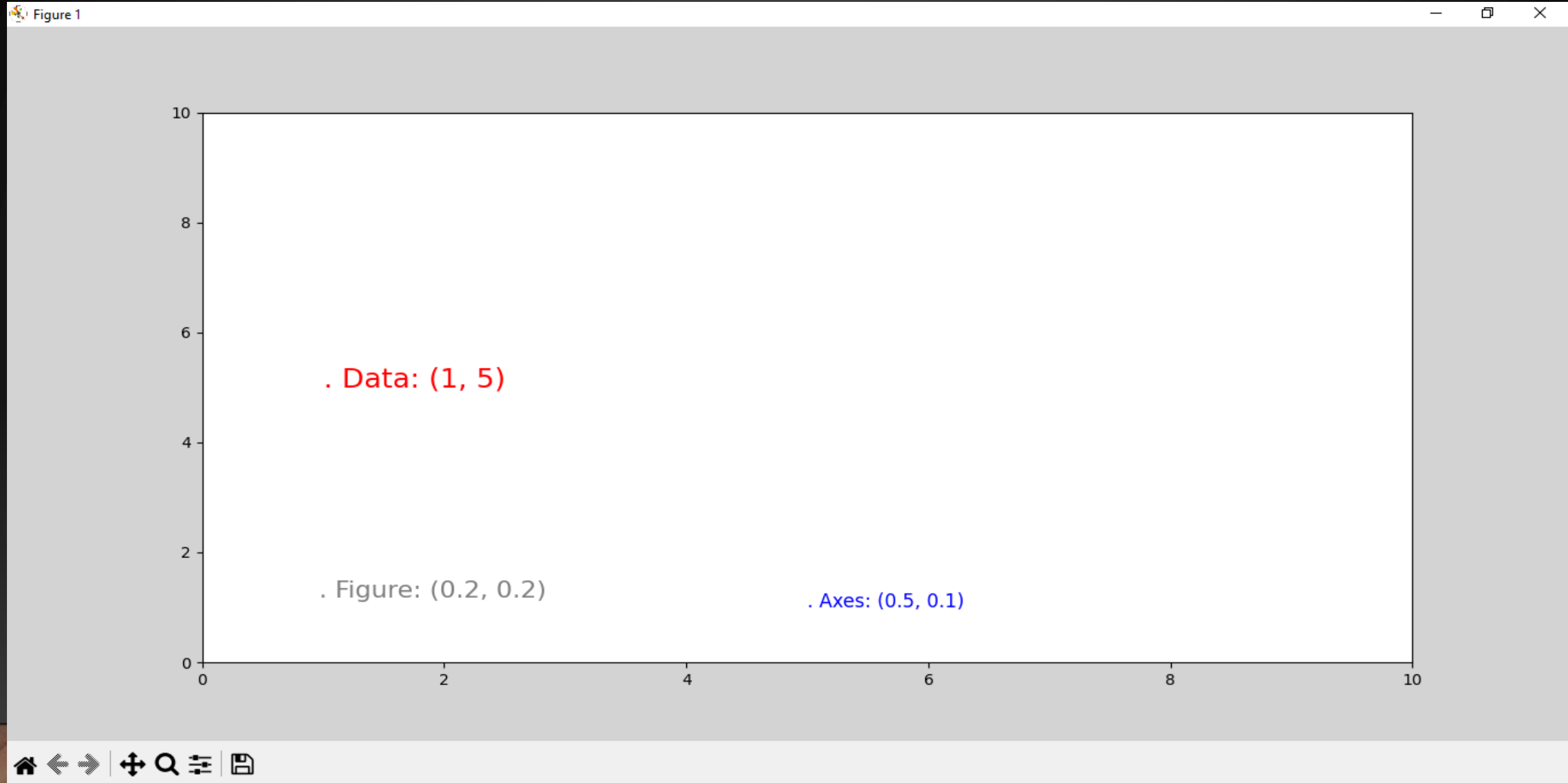
- To anchor the text to a position on the axes or figure, independent of the data → using 'transform' method.
- Three predefined transforms:
- **ax.transData** → default → Transform associated with data coordinates → The transData coordinates → give the usual data coordinates associated with the x- and y-axis labels.
- **ax.transAxes** → Transform associated with the axes (in units of axes dimensions) → The transAxes coordinates → give the location from the bottom-left corner of the axes as a fraction of the axes size.
- **fig.transfigure** → Transform associated with the figure (in units of figure dimensions) → The transfigure coordinates → specify the position from the bottom left of the figure as a fraction of the figure size.

TRANSFORMS AND TEXT POSITION

```
import matplotlib.pyplot as plt
import numpy as np
fig, ax = plt.subplots(facecolor='lightgray')
ax.axis([0, 10, 0, 10])

ax.text(1, 5, ". Data: (1, 5)", transform=ax.transData, fontsize=18, color='red')
ax.text(0.5, 0.1, ". Axes: (0.5, 0.1)", transform=ax.transAxes, fontsize=12,
color='blue')
ax.text(0.2, 0.2, ". Figure: (0.2, 0.2)", transform=fig.transFigure, fontsize=16,
color='gray');
plt.show()
```

TRANSFORMS AND TEXT POSITION



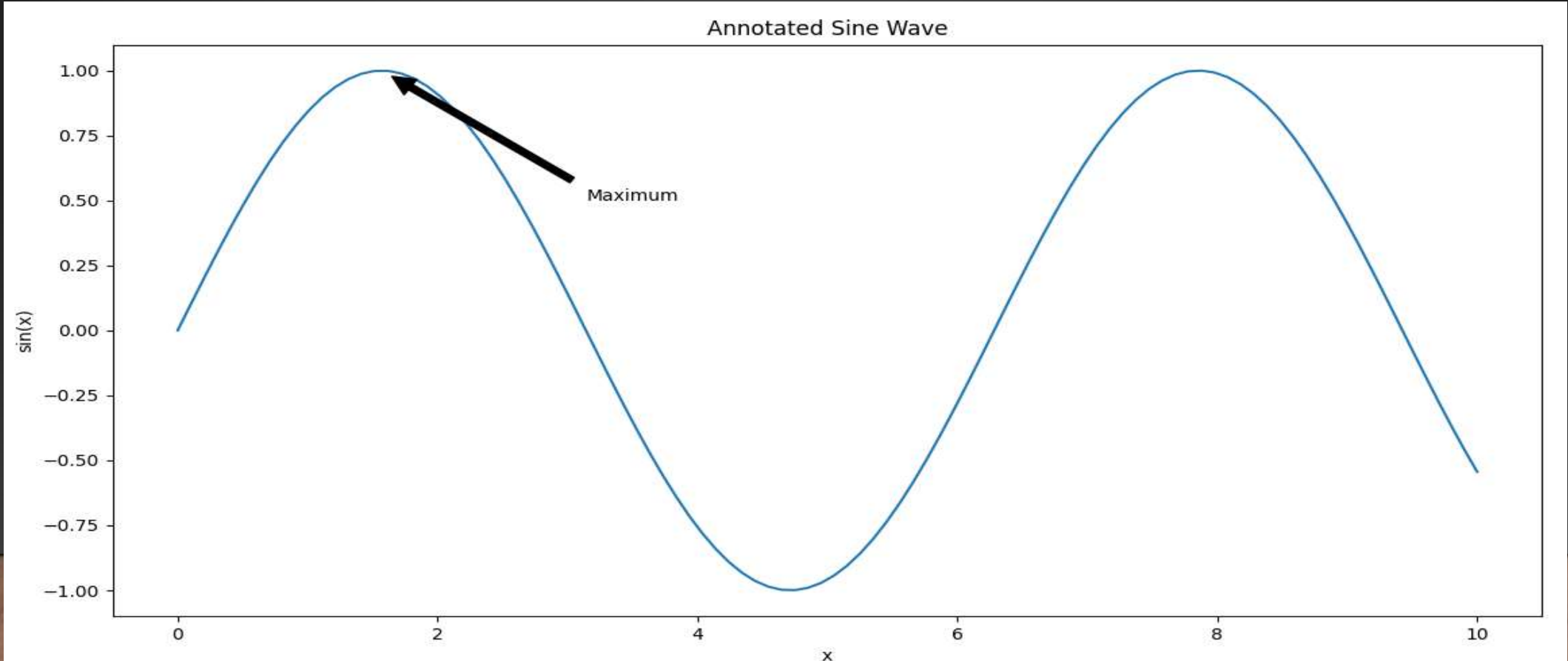
ARROWS AND ANNOTATION

- Along with tick marks and text, another useful annotation mark is the simple arrow $\rightarrow$ drawn using the `plt.annotate()` function $\rightarrow$ this function creates some text and an arrow, and the arrows can be very flexibly specified.

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.title('Annotated Sine Wave')
plt.annotate('Maximum', xy=(np.pi / 2, 1), xytext=(np.pi, 0.5),
arrowprops=dict(facecolor='black', shrink=0.05))
plt.show()
```

ARROWS AND ANNOTATION

- Along with tick marks and text, another useful annotation mark is the simple arrow $\rightarrow$ drawn using the `plt.annotate()` function $\rightarrow$ this function creates some text and an arrow, and the arrows can be very flexibly specified.

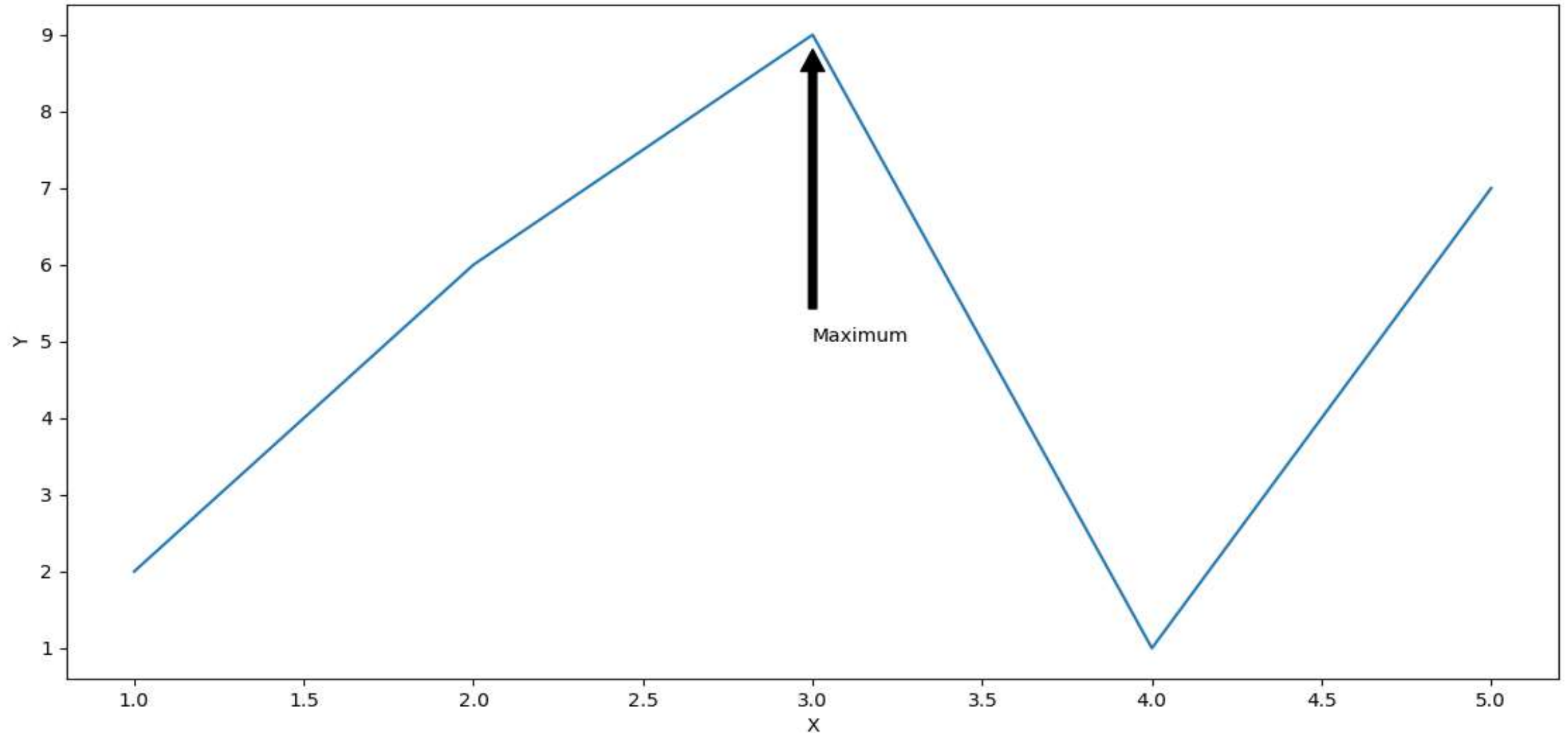


ARROWS AND ANNOTATION

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 6, 9, 1, 7])
plt.plot(x, y)
plt.annotate('Maximum', xy=(x[np.argmax(y)], max(y)), xytext=(3, 5),
arrowprops=dict(facecolor='black', shrink=0.05))
plt.xlabel('X')
plt.ylabel('Y')
plt.title('A Sample Annotated Graph')
plt.show()
```

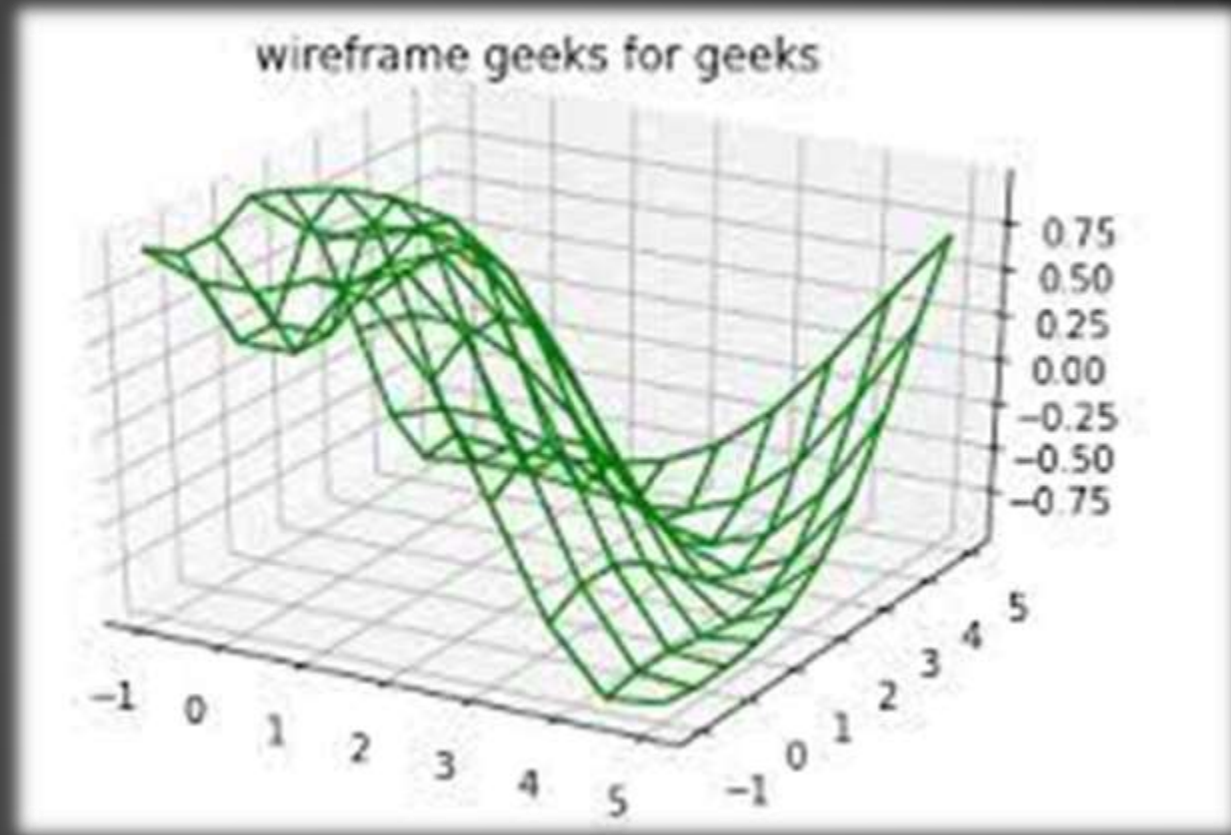
ARROWS AND ANNOTATION

A Sample Annotated Graph



THREE-DIMENSIONAL PLOTTING

- **PLT.AXES(PROJECTION = '3D')**
- **PLT.FIGURE()**
- **AX.PLOT3D()**
- **AX.SCATTER3D()**
- **AX.CONTOUR3D()**
- **AX.PLOT_WIREFRAME()**
- **NP.MESHGRID()**
- **AX.PLOT_SURFACE(**



THREE-DIMENSIONAL PLOTTING

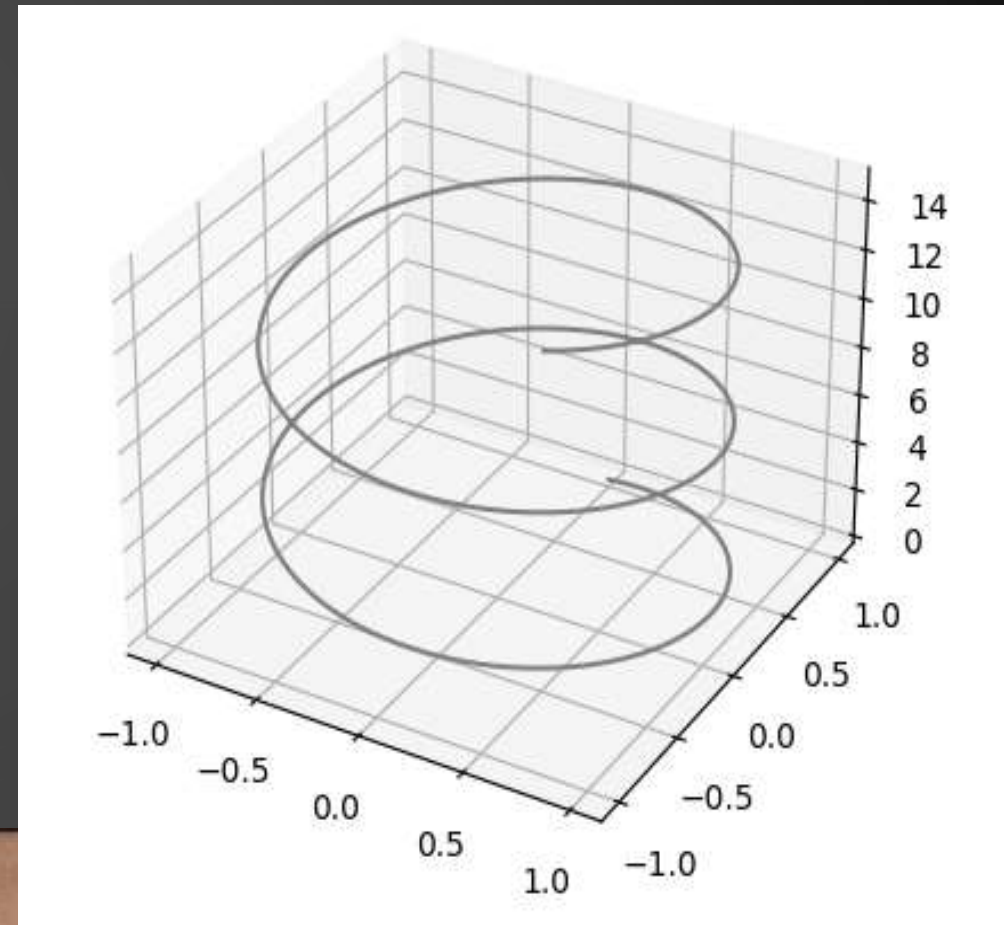
- Three-dimensional plots can be created by importing the mplot3d toolkit, included with the main Matplotlib installation

```
from mpl_toolkits import mplot3d
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
ax = plt.axes(projection='3d')
```

THREE-DIMENSIONAL POINTS AND LINES

- Three-dimensional plots can be created from sets of (x,y, z) triples.
- Eg. Data for a three-dimensional line

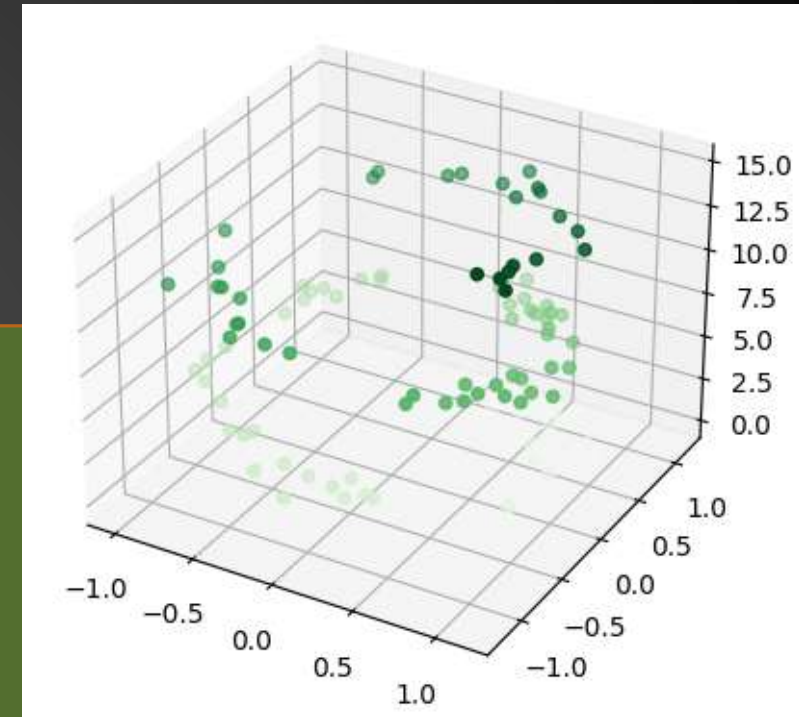
```
from mpl_toolkits import mplot3d
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
ax = plt.axes(projection='3d')
zline = np.linspace(0, 15, 1000)
xline = np.sin(zline)
yline = np.cos(zline)
ax.plot3D(xline, yline, zline, 'gray')
plt.show()
```



THREE-DIMENSIONAL POINTS AND LINES

- Three-dimensional plots can be created from sets of (x,y, z) triples.
- Eg. Data for three-dimensional scattered points

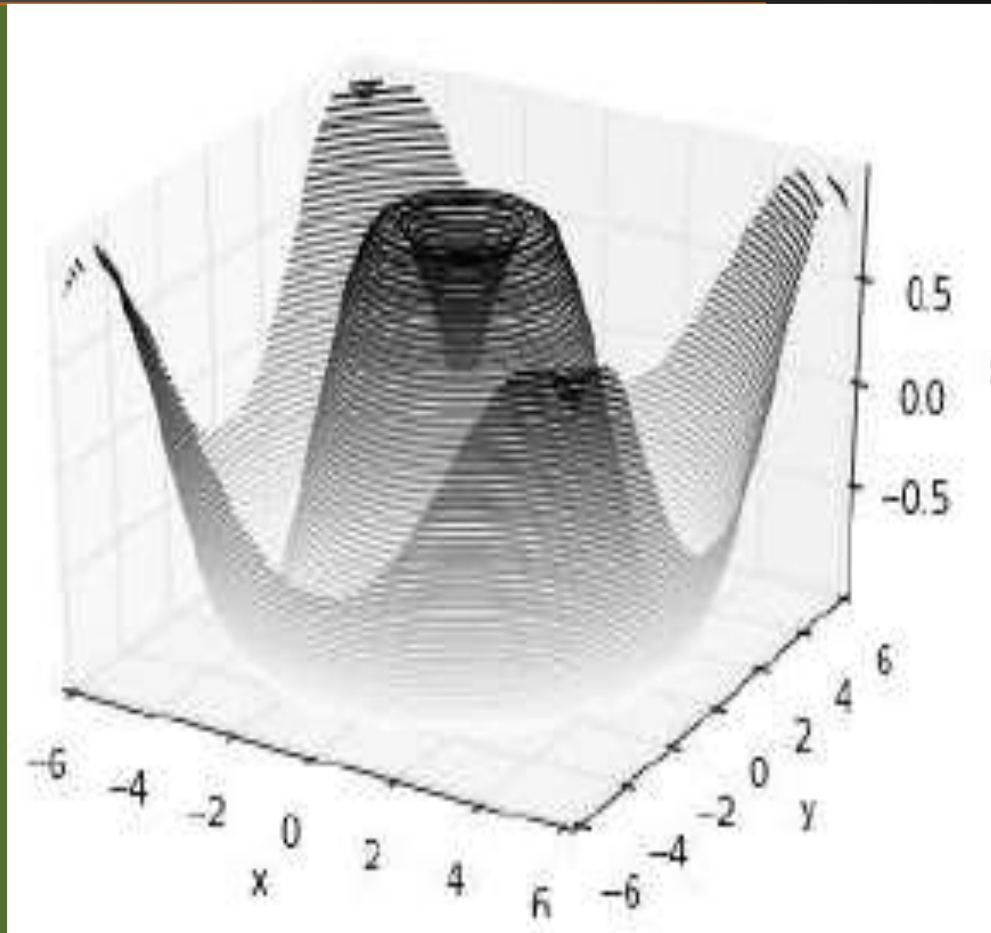
```
from mpl_toolkits import mplot3d
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
ax = plt.axes(projection='3d')
xdata = np.sin(zdata) + 0.1 * np.random.randn(100)
ydata = np.cos(zdata) + 0.1 * np.random.randn(100)
zdata = 15 * np.random.random(100)
ax.scatter3D(xdata, ydata, zdata, c=zdata, cmap='Greens')
plt.show()
```



THREE-DIMENSIONAL CONTOUR PLOTS

- Three-dimensional plots can be created from sets of (x,y, z) triples.
- Eg. Data for three-dimensional scattered points

```
x = np.linspace(-6, 6, 30)
y = np.linspace(-6, 6, 30)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X ** 2 + Y ** 2))
fig = plt.figure()
ax = plt.axes(projection='3d')
ax.contour3D(X, Y, Z, 50, cmap='binary')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
```



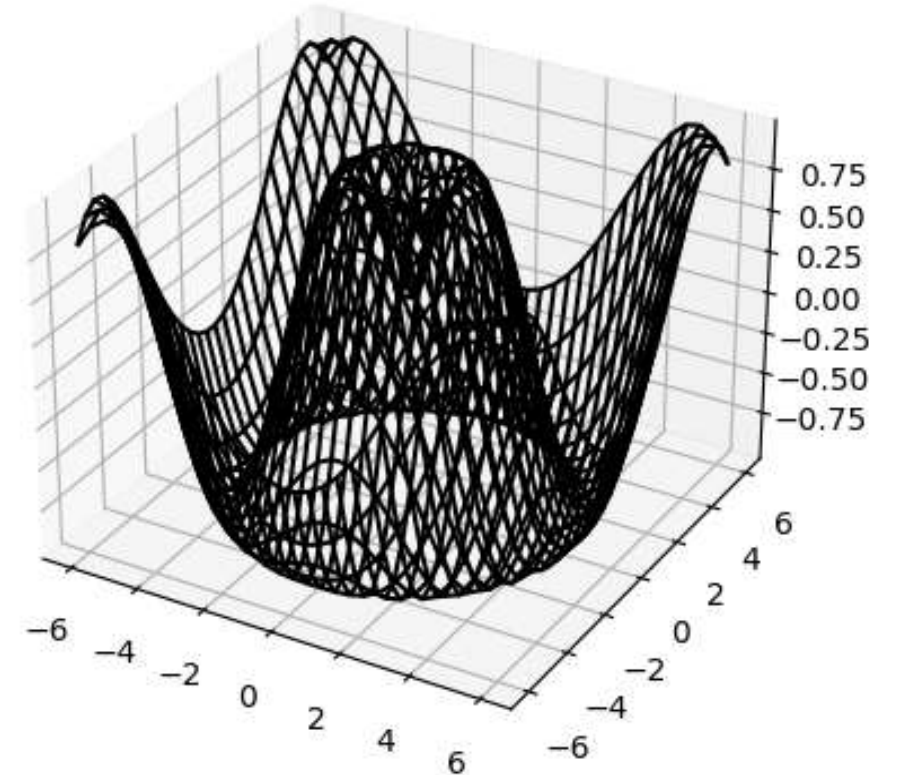
WIREFRAMES AND SURFACE PLOTS

- Two other types of three-dimensional plots that work on gridded data are
 - Wireframes and
 - Surface plots.
- These take a grid of values and project it onto the specified three-dimensional surface, and can make the resulting three-dimensional forms quite easy to visualize.

WIREFRAMES AND SURFACE PLOTS

- Wireframes

```
x = np.linspace(-6, 6, 30)
y = np.linspace(-6, 6, 30)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X ** 2 + Y ** 2))
fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_wireframe(X, Y, Z, color='black')
ax.set_title('wireframe')
plt.show()
```



WIREFRAMES AND SURFACE PLOTS

- Surface plots – it is like a wireframe plot, but each face of the wireframe is a filled polygon.

```
x = np.linspace(-6, 6, 30)
y = np.linspace(-6, 6, 30)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X ** 2 + Y ** 2))
fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1, cmap='viridis',
edgecolor='none')
ax.set_title('surface');plt.show()
```

